

Gen AI Engineer

Interview Preparation - Detailed Study Guide

Candidate: Joemon Jesudas

Target: Gen AI Engineer Role

Prep Window: 5 Days

Prepared: 19 May 2026

Suggested 5-Day Study Plan

Day	Focus Area	Goal
Day 1	NLP fundamentals + Transformers	Build the language-model foundation
Day 2	Generative AI (text/image/audio, diffusion, fine-tuning)	Understand model families & training paradigms
Day 3	Agentic AI + Frameworks (LangGraph, ADK, AutoGen) + MCP	Master agent design and orchestration
Day 4	GenAI app delivery + Data/Feature engineering at scale	Tie it to production
Day 5	Responsible AI, SWE best practices, embeddings, mock Q&A	Polish and rehearse

Table of Contents

- 1. Natural Language Processing (NLP)
- 2. Generative AI Concepts
- 3. Agentic AI - Concepts and Real-World Applications
- 4. Agentic AI Frameworks: LangGraph, ADK, AutoGen
- 5. Developing & Deploying GenAI Applications
- 6. Data Processing, Feature Engineering & Training at Scale
- 7. Responsible AI, Ethics, Governance & Risk Mitigation
- 8. Software Engineering Best Practices for ML Systems
- 9. MCP vs Typical Tool Calling Methods
- 10. Embedding Models: text-embedding-ada-002 vs text-embedding-3
- 11. Quick-Recall Cheat Sheet
- 12. Likely Interview Questions

1. Natural Language Processing (NLP)

1.1 What is NLP?

NLP is the branch of AI that enables machines to understand, generate, and reason about human language. Modern NLP is dominated by **transformer-based language models** trained on massive text corpora.

1.2 Core NLP Tasks

Task	Description	Example Use Case
Tokenization	Split text into tokens	Preprocessing before any LM call
POS tagging	Tag tokens (noun, verb, etc.)	Information extraction
NER	Identify entities (people, dates, money)	Claims processing, KYC
Sentiment analysis	Classify emotional tone	Customer feedback, social listening
Text classification	Assign labels	Spam, intent detection
Topic modeling	Discover themes	LDA, BERTopic
Machine Translation	Convert across languages	Multilingual support
Summarization	Compress documents	Policy / contract summarization
Question Answering	Answer from context	RAG-style Q&A
Text Generation	Produce coherent text	Chatbots, drafting
Coreference	Resolve "he/she/it" references	Conversation understanding
Semantic search	Vector-based retrieval	RAG, knowledge bases

1.3 Tokenization

- **Word-level:** simple, but explodes vocab and OOV problems.
- **Character-level:** small vocab, very long sequences.
- **Subword (BPE / WordPiece / SentencePiece):** best of both - used by GPT, BERT, T5, Llama.
- Always know: tokens != words. ~100 tokens = ~75 English words.

1.4 Embeddings & Vector Representations

- **Static:** Word2Vec, GloVe, FastText - one vector per word, ignores context.
- **Contextual:** BERT, ELMo, GPT - same word gets different vectors based on context.
- **Sentence/Document embeddings:** Sentence-BERT, OpenAI text-embedding-3.
- **Use cases:** semantic search, clustering, classification, RAG retrieval, deduplication.

1.5 Language Models - Evolution

1. **n-gram models** - count-based; suffer from sparsity.
2. **RNN / LSTM / GRU** - sequential, vanishing gradient issues, slow.
3. **Transformer (2017, "Attention Is All You Need")** - parallelizable, self-attention, the foundation for GPT/BERT/T5/Llama.
4. **Pretrained LMs** - BERT (encoder), GPT (decoder), T5/BART (encoder-decoder).
5. **Large Language Models (LLMs)** - billions of parameters, emergent abilities, instruction-tuned, RLHF-aligned.

1.6 Transformer Architecture (deeper)

- **Self-attention:** each token attends to every other token; captures long-range dependencies. Q, K, V matrices: $\text{Attention} = \text{softmax}(QK^T / \sqrt{d_k}) * V$.

- **Multi-head attention:** multiple heads in parallel learn different relations.
- **Positional encoding:** since attention is permutation-invariant, positions are added (sinusoidal or learned, RoPE in modern LLMs).
- Feed-forward layers, layer norm, residual connections.
- **Encoder-only (BERT):** good for understanding/classification.
- **Decoder-only (GPT):** good for generation; uses causal masking.
- **Encoder-decoder (T5, BART):** translation, summarization.

1.7 Sentiment Analysis - How It is Done

1. **Lexicon-based:** VADER, TextBlob - fast, no training, weak on sarcasm.
2. **Classical ML:** TF-IDF + Logistic Regression / SVM / Naive Bayes - strong baseline.
3. **Deep learning (LSTM/CNN):** captures sequence context.
4. **Transformer fine-tuning:** BERT/RobERTa fine-tuned on labeled sentiment data - current default for accuracy.
5. **Zero-shot via LLMs:** prompt GPT-4o / Claude with classes - no training needed, great for cold start.
6. **Aspect-Based Sentiment Analysis (ABSA):** sentiment per aspect ("battery: positive, screen: negative").

Pitfalls to flag in interviews: sarcasm, negation, domain shift, class imbalance, multilingual, label noise.

1.8 Text Generation - Decoding Strategies

- **Greedy:** pick argmax - repetitive, deterministic.
- **Beam search:** top-k beams kept - good for translation, can be bland.
- **Top-k sampling:** sample from top-k tokens.
- **Top-p (nucleus) sampling:** sample from smallest set whose cumulative prob $\geq p$.
- **Temperature:** $T < 1$ sharper, $T > 1$ more random; $T = 0 \sim$ greedy.
- Repetition penalty / no-repeat-n-gram for cleaner output.

1.9 Evaluation Metrics

- **Classification:** accuracy, precision, recall, F1, AUC.
- **Translation:** BLEU, METEOR, chrF, COMET (learned).
- **Summarization:** ROUGE-1/2/L, BERTScore.
- **Generation quality:** perplexity; BLEU/ROUGE limited; use LLM-as-a-judge / human eval.
- **RAG:** faithfulness, answer relevance, context precision/recall (Ragas, TruLens).

2. Generative AI Concepts

2.1 What is Generative AI?

GenAI produces **new** content - text, image, audio, video, code - by learning the distribution of training data and sampling from it.

2.2 Modalities

Text Generation

- **Models:** GPT-4 / 4o, Claude (Sonnet/Opus/Haiku), Llama 3, Gemini, Mistral.
- **Use cases:** drafting, summarization, chat, code, agents.

Image Synthesis

- **Diffusion:** Stable Diffusion, SDXL, DALL-E 3, Midjourney, Imagen.
- **GANs (older):** StyleGAN.
- **VAEs:** smoother latent space, used inside many diffusion stacks.

Audio Synthesis

- **TTS:** ElevenLabs, OpenAI TTS, Azure Neural TTS - text -> speech.
- **Speech-to-text:** Whisper (OpenAI).
- **Music:** MusicLM, Suno.

Video & Multimodal

- Sora, Veo, Runway. Often diffusion-based with temporal conditioning.

2.3 Diffusion Models

- **Forward process:** progressively add Gaussian noise to data over T steps.
- **Reverse process:** train a UNet to predict noise; iteratively denoise to generate samples.
- **Conditioning:** classifier-free guidance (CFG) lets text prompts steer generation.
- **Latent Diffusion (Stable Diffusion):** denoise in compressed latent space (via VAE) - much cheaper than pixel space.
- Compared with GANs: better mode coverage, more stable training, slower inference (mitigated by DDIM, LCM, distillation).

2.4 Transformers (Recap, in GenAI context)

- Universal backbone for text, code, image (ViT), audio (Whisper) and even video.
- **Why dominant:** parallelism, scalability with data and compute (scaling laws), works across modalities.

2.5 Training Paradigms

1. **Pretraining (self-supervised)** - next-token prediction (causal LM) or masked LM (BERT).
2. **Supervised fine-tuning (SFT)** - instruction-following on (prompt, response) pairs.
3. **RLHF / DPO** - align outputs with human preferences.
4. **Continued pretraining** - domain adaptation on large unlabeled corpora.
5. **PEFT (Parameter-Efficient Fine-Tuning):** LoRA (low-rank adapters), QLoRA (LoRA + 4-bit quantized base), prefix/prompt tuning, IA3.

2.6 Prompt Engineering Toolbox

- Zero-shot, few-shot, chain-of-thought (CoT), self-consistency, ReAct.
- Role / system prompts for persona and constraints.
- Output structuring via JSON schema, function calling, tool use.
- Guardrails: content filtering, output validation, hallucination checks.

2.7 Retrieval-Augmented Generation (RAG)

- **Why:** ground LLM in private/up-to-date data, reduce hallucinations.
- **Pipeline:** ingest -> chunk -> embed -> store (vector DB) -> retrieve -> rerank -> augment prompt -> generate.
- **Advanced:** hybrid search (BM25 + dense), HyDE, query rewriting, parent-document retrieval, multi-vector, GraphRAG, self-RAG.

2.8 Hallucinations - Causes & Mitigations

- **Causes:** missing knowledge, ambiguous prompts, decoding randomness, retrieval misses.
- **Mitigations:** RAG with citations, lower temperature, structured outputs, output validation, evals (Ragas), human-in-the-loop, fine-tuning with grounded data.

3. Agentic AI - Concepts and Real-World Applications

3.1 What is Agentic AI?

An LLM-powered system that can **plan**, **decide**, and **act** by invoking tools / APIs / other agents in a loop, often with memory. It moves beyond single prompt-response to multi-step, goal-directed behavior.

3.2 Core Components of an Agent

1. **LLM (the brain)** - reasoning and planning.
2. **Tools** - search, calculators, DB queries, APIs, code execution, browser.
3. **Memory** - short-term (context window), long-term (vector DB), episodic (conversation history).
4. **Planner** - decomposes goal into sub-tasks.
5. **Executor / Controller** - runs tools, handles errors, retries.
6. **Reflection / Critic** - self-evaluates and replans.

3.3 Agent Patterns

- **ReAct (Reason + Act)**: thought -> action -> observation -> repeat.
- **Plan-and-Execute**: make a full plan, then execute step-by-step.
- **Reflexion**: agent critiques its own outputs and retries.
- **Tree of Thoughts**: explore multiple reasoning branches.
- **Multi-agent collaboration**: specialized agents (planner, coder, reviewer, executor) talk to each other.
- **Hierarchical agents**: supervisor delegates to worker agents.

3.4 Real-World Use Cases

Domain	Agent Use Case
Insurance	Claims triage agent - reads FNOL, fetches policy, checks fraud signals, drafts letter
Customer support	Multi-turn troubleshooting + ticket creation
Sales / CRM	Research lead -> draft email -> schedule -> log to CRM
Coding	SWE-agent / Devin - clone repo, fix bug, run tests, open PR
Data analysis	NL -> SQL -> chart -> narrative
Research	Web search + synthesize report
RPA replacement	Browser automation across legacy enterprise apps

3.5 Designing a Robust Production Agent

- **Bound the action space**. Whitelist tools and scopes.
- **Deterministic shells around the LLM** - validate inputs/outputs; never trust raw LLM output for irreversible actions.
- **Step / token / cost budgets**. Hard caps to prevent runaway loops.
- **Observability** - log every thought, tool call, observation; capture traces (LangSmith, Langfuse, OpenTelemetry).
- **Eval harness** - task-level pass/fail, intermediate-step quality, regression suites.
- **Human-in-the-loop checkpoints** for high-risk actions (sending money, deletes, emails).
- **Idempotency & retries**.
- **Failure modes to plan for**: infinite loops, wrong tool choice, tool flakiness, prompt injection, jailbreaks.

3.6 Prompt Injection / Tool Misuse

- Treat any retrieved document or tool output as **untrusted user input**.
- Mitigations: separation of system / user / data prompts, allowlisted tools, output filters, rate limits.

4. Agentic AI Frameworks: LangGraph, ADK, AutoGen

4.1 LangGraph (LangChain)

- **What:** library for building **stateful, graph-based agent workflows** as directed (often cyclic) graphs.
- **Mental model:** nodes = functions/agents, edges = transitions, state = a typed dict that flows through the graph.
- **Why graphs over chains?** Chains are linear; agents need branching, loops, retries, and human-in-the-loop - graphs express that naturally.
- **Key features:** persistent state (checkpointers), streaming, time-travel/replay, human-in-the-loop interrupts, native multi-agent topologies.
- **When to pick:** complex flows with branching/looping, long-running stateful processes, need for resumability.
- **Drawbacks:** more boilerplate than simple chains; ecosystem-coupled to LangChain.

4.2 Google ADK (Agent Development Kit)

- **What:** Google open-source framework for building, evaluating, deploying agents on Gemini and Vertex AI Agent Engine.
- First-class **multi-agent hierarchies** (Sequential, Parallel, Loop agents).
- Built-in tool ecosystem and easy Gemini function-calling.
- Tight integration with **Vertex AI Agent Engine** for managed deployment.
- Eval framework and tracing baked in.
- **When to pick:** Google Cloud / Gemini stack, want managed deployment, multi-agent pipelines with strict orchestration semantics.

4.3 AutoGen (Microsoft)

- **What:** Microsoft Research framework for **multi-agent conversation**. Agents (e.g., AssistantAgent, UserProxyAgent) chat with each other to solve tasks.
- "Conversation programming" - define agents and let them converse.
- Built-in code-execution agent.
- GroupChat, RoundRobin, Selector orchestration.
- AutoGen Studio (low-code UI). v0.4+ moved to event-driven async actor model.
- **When to pick:** problems naturally framed as conversations between specialists (planner <-> coder <-> reviewer), code-generation/automation.

4.4 Quick Comparison

Aspect	LangGraph	Google ADK	AutoGen
Vendor	LangChain	Google	Microsoft
Paradigm	Stateful graph	Hierarchical multi-agent	Multi-agent conversations
Best for	Complex stateful flows	GCP/Gemini agents	Conversational + code exec
State	Checkpointers	Sessions, memory services	Conversation history
Deployment	LangGraph Cloud / self-host	Vertex AI Agent Engine	Self-host / Azure
Strength	Control + observability	Tight GCP integration	Easy multi-agent chats
Weakness	LC ecosystem coupling	Best on Google stack	Less branch control

4.5 Other Names Worth Knowing

- **CrewAI** - role-based crews of agents.
- **OpenAI Swarm / Agents SDK** - lightweight handoffs.
- **LlamaIndex Agents** - retrieval-centric.
- **Semantic Kernel** (Microsoft) - plugins and planners, .NET-friendly.
- **Haystack Agents** - retrieval-first pipelines.

5. Developing & Deploying GenAI Applications

5.1 Application Archetypes

Archetype	Examples
Chat / Conversational AI	Copilot, claim handler assistant
Document understanding	Policy / contract summarization, extraction
RAG knowledge assistant	Internal Q&A over wiki / docs
Workflow agents	Claims triage, code reviewer
Image synthesis / editing	Marketing assets, design
Voice agents	IVR, call summarization
Code assistants	Repo-aware code generation

5.2 Reference Architecture

```

[ Client / Frontend (Streamlit / React) ]
    |
    v
[ FastAPI Service Layer ] -- auth, rate limit, RBAC
    |
    +-----+-----+
    |               |
    v               v
[ Orchestrator ] [ Tools / APIs ]
(LangGraph / Agents)
    |
    v
[ LLM Gateway (Azure OpenAI) ]
    |
    v
[ Vector Store (PGVector) ] [ Object / Doc Store ]
    |
    v
[ Observability: Grafana, MLflow, Evidently, traces ]

```

5.3 Lifecycle

1. Define use case + KPIs (latency, accuracy, cost/request, CSAT).
2. Data prep & evals - gather golden datasets *first*.
3. Prompt + retrieval prototyping - Jupyter / Streamlit POC.
4. Hardening - guardrails, schema validation, retries, fallbacks.
5. API + UI layer - FastAPI + Streamlit / Web.
6. Containerize - Docker; orchestrate with Kubernetes + Helm.
7. CI/CD - GitHub Actions; lint, tests, eval gate, build, deploy.
8. Observability - logs, traces, token/cost metrics, drift, eval scores.
9. Continuous improvement - feedback loops, prompt versioning, fine-tuning.

5.4 Latency, Cost, and Reliability Levers

- **Streaming** responses to cut perceived latency.
- **Smaller / cheaper model first** (Haiku/Sonnet/4o-mini), fallback to bigger model only when needed (model

cascading).

- **Caching** - semantic cache for prompts; LLM gateway caches.
- **Batching embeddings**.
- **Retrieval first, generation second** - avoid huge prompts.
- **Quantization & distillation** for self-hosted models.
- **Async + retries with exponential backoff**, circuit breakers.

5.5 Evaluation in Production

- **Offline**: golden sets, Ragas (faithfulness, context precision/recall), task-specific accuracy.
- **Online**: A/B tests, thumbs-up/down, regression suites on each prompt/model change.
- **Drift**: input drift (Evidently), output drift, retrieval-quality drift.

6. Data Processing, Feature Engineering & Training at Scale

6.1 Data Sources & Ingestion

- Files (PDFs, scans), databases (SQL/NoSQL), event streams (Kafka), data lakes (Azure Data Lake, S3), APIs, SaaS (CRM, ticketing).
- **Tools:** Airbyte, Fivetran, custom Airflow / dbt pipelines, PySpark for large data.

6.2 Data Cleaning & Preparation

- Deduplication, missing value handling (mean/median/predictive/dropping).
- Outlier handling (IQR, z-score, domain rules).
- Text cleaning (Unicode normalization, language detection, PII redaction).
- Image cleaning (resizing, denoise, augmentation).
- Schema validation (Pydantic, Great Expectations).

6.3 Feature Engineering

Tabular

- Encoding - one-hot, target, frequency, ordinal.
- Scaling - StandardScaler, MinMax, RobustScaler.
- Interactions, ratios, lags, rolling stats (time series).
- Datetime decomposition (year/month/day/dow/holiday flags).
- Binning, log/Box-Cox transforms.

Text

- TF-IDF, n-grams (classical baselines).
- Embeddings (OpenAI, Sentence-BERT) for semantic features.
- Topic features (LDA/BERTopic).

Image

- Pretrained CNN / ViT embeddings.
- Data augmentation (flip, crop, color jitter, mixup, cutmix).

Feature Stores

- Feast, Tecton - consistent training/serving features, point-in-time correctness.

6.4 Training on Large Datasets

- **Distributed training:** data parallelism (DDP, Horovod), model/tensor parallelism (Megatron, FSDP), pipeline parallelism, ZeRO/FSDP.
- **Mixed precision (fp16/bf16)** - speed + memory wins.
- **Gradient accumulation** for effective large batch sizes.
- **Checkpointing** - gradient checkpointing for memory; checkpoint files for fault tolerance.
- **PEFT (LoRA/QLoRA)** for fine-tuning huge models cheaply.
- **Hyperparameter tuning:** grid, random, Bayesian (Optuna), HyperBand, ASHA.
- **Frameworks:** PyTorch Lightning, HuggingFace Accelerate / TRL, DeepSpeed, Ray Train.

6.5 Reproducibility

- DVC for data + model versioning.
- MLflow / Weights & Biases for experiment tracking.
- Pinned environments (poetry/uv, conda-lock, Docker).
- Deterministic seeds where feasible.

7. Responsible AI, Ethics, Governance & Risk Mitigation

7.1 Core Pillars

1. **Fairness** - no disparate harm across protected groups.
2. **Transparency / Explainability** - model decisions can be understood.
3. **Privacy** - data minimization, anonymization, consent.
4. **Security** - protect models, data, prompts from attack.
5. **Safety** - avoid harmful outputs (toxicity, bias, illegal content).
6. **Accountability** - clear ownership, audit trails.
7. **Reliability** - performs as intended under realistic conditions.
8. **Inclusiveness** - works across demographics, languages, accessibility needs.

7.2 Bias and Fairness

- **Sources:** biased data, biased labels, proxy variables, sample imbalance, deployment context.
- **Metrics:** demographic parity, equal opportunity, equalized odds, disparate impact.
- **Mitigations:** rebalancing, reweighting, counterfactual augmentation, adversarial debiasing, post-processing thresholds.

7.3 Explainability

- **Global:** feature importance, partial dependence.
- **Local:** SHAP, LIME, integrated gradients.
- **For LLMs:** chain-of-thought traces (with caveats), attention visualization (limited), citation-grounded answers.

7.4 Privacy

- PII detection & redaction (Presidio, Azure Language).
- Differential privacy (DP-SGD).
- Federated learning when data cannot move.
- GDPR, HIPAA, India DPDP, EU AI Act, ISO 42001 awareness.

7.5 LLM-Specific Risks

- Hallucination, prompt injection, jailbreaks, data leakage, training-data memorization, copyright, toxic output, over-reliance.
- **Mitigations:** RAG with citations, strict system prompts, input/output classifiers (Azure Content Safety, Llama Guard), allow-listing, rate limits, red-teaming.

7.6 Model Governance

- Model registry (MLflow / Azure ML).
- Model cards & datasheets.
- Approval workflows; promotion gates from dev -> staging -> prod.
- Audit logs (who changed prompt/model/config).
- Version-controlled prompts (treat prompts as code).
- Periodic re-evaluation; sunset criteria.

7.7 Frameworks & Standards (Good Name-Drops)

- **NIST AI Risk Management Framework (AI RMF).**
- **EU AI Act** - risk tiers (unacceptable, high, limited, minimal).
- **ISO/IEC 42001** - AI management systems.
- **Microsoft Responsible AI Standard** (relevant for Allianz on Azure).

8. Software Engineering Best Practices for ML Systems

8.1 Code Quality

- Strong typing (Pydantic, mypy), linting (ruff), formatting (black/ruff-format).
- Small functions, dependency injection, single-responsibility.
- Do not hide complexity in notebooks - production code lives in modules with tests.

8.2 Testing the ML/LLM Stack

- **Unit tests** - pure functions (parsers, validators, prompt builders).
- **Integration tests** - DB, retrieval, model client (with VCR / recorded responses).
- **Eval tests** - accuracy / faithfulness thresholds on golden sets, run in CI.
- **Contract tests** - JSON-schema validation of LLM outputs.
- **Load / latency tests** - locust, k6.
- **Red-team / safety tests** - jailbreak prompts, PII leakage.

8.3 CI/CD

- GitHub Actions / Azure DevOps.
- Stages: lint -> unit -> integration -> eval-gate -> security scan -> build image -> deploy.
- Promote artifacts (image digest + model version + prompt version) - never rebuild for prod.

8.4 Configuration & Secrets

- 12-factor - config in env vars / config service, secrets in Key Vault.
- Feature flags for prompt / model rollouts (gradual rollout, instant rollback).

8.5 Observability

- **Structured logs** (JSON), **traces** (OpenTelemetry / LangSmith / Langfuse), **metrics** (Prometheus + Grafana).
- Track per request: prompt + model + version + tokens in/out + latency + cost + tool calls + outcome + user feedback.

8.6 Reliability & Operations

- Idempotent endpoints; retries with backoff; circuit breakers; bulkheads.
- Graceful degradation (fallback model, cached answer).
- SLOs: latency p95, error rate, eval-score thresholds.
- On-call runbooks for "LLM down", "vector DB down", "drift detected".

8.7 Architecture Principles

- Separate **orchestration** from **business logic** from **model serving**.
- Treat LLM as an unreliable external API - always validate.
- Keep deterministic logic in code, not in prompts.
- Make prompts version-controlled artifacts.
- Prefer composable small services (gateway -> retrieval -> reranker -> generator) over a monolith.

9. MCP vs Typical Tool Calling Methods

9.1 Typical (Native) Tool / Function Calling

LLM provider exposes a feature where the model can output a structured "call this function with these args" instruction. Examples: OpenAI **function calling / tools**, Anthropic **tool_use**, Gemini **function calling**.

Flow

1. App sends user prompt + JSON schemas of available tools to the model.
 2. Model decides whether to call a tool and emits a structured tool_call.
 3. App executes the tool locally, returns the result.
 4. Model continues until final answer.
- **Where it lives:** inside *your* application - every app re-implements its own tool registry, auth, schemas.

Strengths

- Simple to start, low-latency, full control.
- Tightly coupled to one app - easy to optimize.

Weaknesses

- **Per-app reinvention:** every app builds its own connectors (Jira, Confluence, GitHub, DB, ...).
- **Vendor-specific schemas** - switching from OpenAI to Anthropic requires rewriting.
- **Hard to share** tools across teams / agents / IDEs.
- **Auth & sandboxing** are your problem; easy to leak secrets.

9.2 MCP (Model Context Protocol)

An **open protocol** (introduced by Anthropic late 2024; widely adopted in 2025) that standardizes how LLM applications talk to external **tools, resources, and prompts**.

- Architecture: **MCP host** (the LLM app - Claude Desktop, Cursor, Claude Code, your agent) <-> **MCP client** (1 per server) <-> **MCP server** (exposes tools/resources/prompts for a system, e.g., GitHub, Confluence, Jira, Postgres, filesystem).
- Transport: stdio (local) or HTTP/SSE (remote); JSON-RPC under the hood.
- A server exposes three primitive types: **Tools** (model-invoked actions), **Resources** (readable data), **Prompts** (reusable templates).

Strengths

- **Write once, reuse everywhere** - same MCP server works in Claude Desktop, Cursor, Claude Code, any custom agent.
- **Decouples capability from app** - teams ship MCP servers any internal agent can use.
- **Standardized auth, discovery, schemas**.
- **Composable** - host can connect many servers; agent gets a federated tool catalog.
- **Sandboxing & permissions** are first-class.
- Strong ecosystem (official servers for GitHub, Slack, Postgres, Filesystem, Puppeteer, etc.).

Weaknesses / Tradeoffs

- Extra moving piece (the MCP server) - slightly more deployment overhead.
- Latency / extra hop vs in-process calls.
- Standard is still maturing; specs evolve.

- Permission/auth model needs careful design (especially for remote MCP servers).

9.3 Side-by-Side

Dimension	Native Tool Calling	MCP
Standardization	Per-vendor (OpenAI, Anthropic, Gemini)	Open, vendor-neutral
Reusability	Locked to one app	Reusable across hosts
Discovery	Manually listed in code	Server advertises tools/resources/prompts
Auth & sandboxing	DIY	Built into protocol
Best for	Single app, tightly coupled tools, lowest latency	Many agents/apps sharing capabilities; enterprise
Dev overhead	Lower upfront	Slightly higher; pays off at scale
Vendor lock-in risk	Higher	Lower
Composition	Awkward	Designed for it

9.4 When to Use Which (Interview Answer)

- **Use native tool calling** when: small app, few tools, single LLM vendor, ultra-low latency required, no need to share capabilities.
- **Use MCP** when: enterprise setting with many internal systems, multiple agents/IDEs need the same capabilities, avoid vendor lock-in, clean permissioning, plug into Claude Desktop / Cursor / other MCP hosts.
- **In practice you combine them:** an agent uses native tool calling **to invoke MCP-served tools** - MCP standardizes the **catalog and transport**, while the LLM still decides **when** to call each tool.

9.5 In Your Resume Context

You already use MCP at Allianz (model monitoring with MCP + Claude Desktop + Evidently AI). Concrete soundbite:

"We exposed our model-monitoring metrics as an MCP server so Claude Desktop and our internal agents could query drift, performance, and token usage uniformly - instead of writing per-tool function-calling glue in each app."

10. Embedding Models: text-embedding-ada-002 vs text-embedding-3

10.1 Quick Facts

Model	Released	Dimensions	Max Tokens	Notes
text-embedding-ada-002	Dec 2022	1536 (fixed)	8191	Long-time default; superseded
text-embedding-3-small	Jan 2024	1536 default; configurable down	8191	Cheaper *and* better than ada-002
text-embedding-3-large	Jan 2024	3072 default; configurable down	8191	Highest quality; multilingual gains

Note: there is no model literally called "text-ada-embedding-model 003". The successor family is text-embedding-3 (small / large). When an interviewer says "ada 003" they almost always mean the text-embedding-3 family - call this out cleanly; it shows you are current.

10.2 Key Differences

- **Quality:** text-embedding-3-large > 3-small > ada-002 on MIRACL (multilingual) and MTEB (English).
- **Cost:** 3-small is ~5x cheaper than ada-002 while being better.
- **Dimensionality control (3-family only):** Matryoshka Representation Learning lets you **truncate** embeddings (e.g., 3-large@256) with graceful quality degradation. Saves vector DB storage and speeds up search.
- **Multilingual:** v3 is materially better for non-English content.
- **Output normalization:** v3 returns normalized vectors by default - cosine ~ dot product.

10.3 Use Case Mapping

Use Case	Recommended Model	Why
Legacy ada-002 + PGVector @ 1536	Stay on ada-002, or migrate to 3-small (1536)	Same dimensionality avoids reindexing schema changes
New English RAG, cost-sensitive	text-embedding-3-small	Best cost/quality on most English tasks
High-stakes retrieval (legal/medical/insurance), multilingual	text-embedding-3-large	Top quality, especially multilingual
Edge / mobile / huge corpora (storage matters)	3-large truncated to 256 or 512 dims	Matryoshka - keep quality, slash storage
Semantic dedup / clustering at scale	3-small (or 3-large truncated)	Quality + scale balance
Multilingual (DE/FR/IT/ES policies at Allianz)	text-embedding-3-large	Strong cross-lingual performance
Code search	3-large or specialized code embeddings	Larger models capture code semantics better
Legacy compatibility / reproducing prior research	ada-002	Match published baseline

10.4 Migration Notes (ada-002 -> 3-family)

1. **Re-embed your corpus.** Embeddings from different models are not comparable.
2. Decide target dimension; reuse 1536-dim PGVector column with 3-small, or truncate 3-large to 1536.
3. Re-tune similarity thresholds - score distributions change.
4. Re-run retrieval evals (recall@k, MRR, faithfulness in RAG) before flipping.
5. Roll out behind a feature flag; A/B against ada-002 on real queries.

10.5 Likely Trick Question

"Why not just always use text-embedding-3-large?"

Answer: cost, latency, and storage. 3072 dims = 2x the index size and similarity-compute vs 1536. For most English RAG, 3-small at 1536 wins on quality-per-dollar; reserve 3-large for multilingual or accuracy-critical use cases, optionally with Matryoshka truncation.

11. Quick-Recall Cheat Sheet

- **Transformer** = self-attention + FFN + residual + layernorm; positional encodings (sinusoidal/RoPE).
- **BERT** = encoder, MLM. **GPT** = decoder, causal LM. **T5/BART** = encoder-decoder.
- **Decoding**: greedy < beam < top-k < top-p; temperature controls randomness.
- **RAG pipeline**: ingest -> chunk -> embed -> store -> retrieve (+rerank) -> augment -> generate (+cite).
- **Diffusion**: add noise -> train UNet to denoise -> CFG steers with text. Stable Diffusion = latent space.
- **PEFT**: LoRA injects low-rank adapters; QLoRA = LoRA on 4-bit base.
- **Agent loop**: Thought -> Action -> Observation -> repeat until done; bound steps + cost.
- **LangGraph** = stateful graph; **ADK** = Google multi-agent + Vertex; **AutoGen** = multi-agent conversation.
- **MCP** = open protocol; standardizes tool/resource/prompt sharing across apps.
- **Embeddings**: ada-002 (legacy 1536), text-embedding-3-small (1536, cheaper+better), 3-large (3072, multilingual, top quality, truncatable).
- **Responsible AI pillars**: fairness, transparency, privacy, security, safety, accountability, reliability, inclusiveness.
- **MLOps**: registry, CI/CD, eval gates, monitoring (drift + perf + cost), prompt versioning.
- **Top LLM risks**: hallucination, prompt injection, data leakage, copyright, toxic output, over-reliance.

12. Likely Interview Questions

NLP & GenAI

1. Walk me through the transformer architecture. Why did it replace RNNs?
2. Difference between BERT and GPT - when would you pick each?
3. Explain self-attention with the Q/K/V formula.
4. How does sampling temperature affect output? Difference between top-k and top-p?
5. What is tokenization, and why are tokens not words? Implications for cost and context windows.
6. How do diffusion models work, and how does Stable Diffusion differ from pixel-space diffusion?
7. SFT vs RLHF vs DPO - what problems do they each solve?
8. What is LoRA / QLoRA and why does it matter?

RAG & Embeddings

9. Walk through a production RAG pipeline; where do quality issues most often come from?
10. How would you evaluate a RAG system?
11. ada-002 vs text-embedding-3 - when do you use which?
12. How would you migrate an existing ada-002 index to text-embedding-3?
13. Hybrid search - why combine BM25 with vector search?
14. How do you reduce hallucinations in a RAG chatbot for an insurance company?

Agentic AI

15. Define an "agent" precisely. What makes a system "agentic" vs a chatbot?
16. Explain ReAct. What goes wrong with naive ReAct loops in production?
17. Compare LangGraph, Google ADK, and AutoGen - when do you pick each?
18. How do you keep an agent from running away (cost/loops)?
19. How do you protect an agent from prompt injection from retrieved documents?
20. How would you design an agent for claims triage at Allianz?

MCP vs Tool Calling

21. What is MCP and why does it exist?
22. When would native function calling be enough? When does MCP pay off?
23. What primitives does an MCP server expose?
24. How would you secure an MCP server in an enterprise?

Data, Training, MLOps

25. How do you train a transformer on data that does not fit on one GPU?
26. Explain DDP vs FSDP/ZeRO.
27. How do you build reproducible ML pipelines? (DVC, MLflow, Docker, pinned env.)
28. What does a CI/CD pipeline for an LLM app include beyond a typical web app? (eval gate, prompt versioning, safety scans.)
29. What metrics do you put on a Grafana dashboard for an LLM app?

Responsible AI

30. What are your top safety controls for an LLM app facing customers?
31. How do you measure fairness in a claims model?
32. How do you handle PII in prompts and logs?
33. Explain model governance - what artifacts do you version?

Software Engineering

34. How do you test an LLM-powered service?
35. How do you roll out a new prompt or model to production safely?
36. Where do you draw the line between "logic in prompt" and "logic in code"?
37. How do you design for graceful degradation when the LLM provider is down?

Final Tips for the 5 Days

- Talk in concrete numbers from your resume: 10,000+ policy docs, sub-second retrieval, 30% decision-consistency lift, 5M GBP savings. Anchor every answer in a story.
- STAR format (Situation, Task, Action, Result) for behavioral and project questions.
- Whiteboard one architecture end-to-end (e.g., the liability prediction RAG). Be ready to draw it.
- Have one strong opinion per topic ("for English RAG I default to text-embedding-3-small unless multilingual is needed; then 3-large with Matryoshka truncation").
- Practice a 60-second pitch about yourself, plus a 2-minute deep-dive on your most impressive project.
- Day before: rest, skim the cheat sheet, review your stories. Do not cram.

Good luck!